DATA STRUCTURES

Q1) When to use Linked List?

When we want to work with an unknown number of data values, we use a linked list data structure to organize that data.

Q2) What is Linked List?

The linked list is a linear data structure that contains a sequence of elements such that each element links to its next element in the sequence. Each element in a linked list is called "Node".

Q3) What is Single Linked List?

Simply a list is a sequence of data, and the linked list is a sequence of data linked with each other.

The formal definition of a single linked list is as follows...

-Single linked list is a sequence of elements in which every element has link to its next element in the sequence.

Q4)What is Circular Linked List?

A circular linked list is a sequence of elements in which every element has a link to its next element in the sequence and the last element has a link to the first element.

Q5)What is Doubly linked list?

A doubly linked list is a type of linked list where each node has two pointers: a prev pointer and a next pointer. The prev pointer points to the previous node in the list, while the next pointer points to the next node in the list. This allows for efficient access to both the previous and next nodes, making it useful for a variety of applications such as reversing a linked list, implementing a queue, and implementing a stack.

Q6) Important Points to be remembered:

In a single linked list, the address of the first node is always stored in a reference node known as "front"(Some times it is also known as "head").

Always next part (reference part) of the last node must be NULL.

Q7) Operations on Single Linked List

The following operations are performed on a Single Linked List

•Insertion

•Deletion

•Display

LINKED LIST

INTRODUCTION:

✓ It is a collection of linear list of data elements.

✓ The data elements are called **nodes.**

✓ Each node contains **two** parts: **data** and **link**.

✓ The data represents **integers** and link is a **pointer** that points to next node.

✓ The last node of the linked list is not connected to any node so it stores the value NULL in link part.

✓ Here **NULL** is defined as -1

✓ NULL pointer denotes **end** of the list.

✓ It contains pointer variable called start node that contains the address of first node in the list

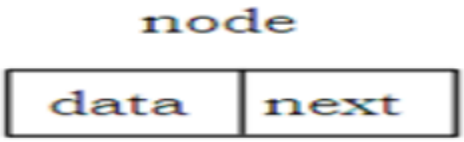
✓ We can traverse the list starting from start node that contains first node address and in turn first node contains second node address and so on thus forming chain of nodes.

✓ If **start == NULL** then the list is empty.

✓ The diagrammatic representation of linked list is shown below

Simple linked list :

✓ Representation of a node



✓In C language, the code for the linked list is

```
struct node
{
        int data;
        struct node *next;
}
```

SINGLE LINKED LIST AND ITS REPRESENTATION IN MEMORY:

✓ **"A single linked list is a linked list in which each node contains only one link pointing to the next node in the list"**.

✓ A linked list allocates space for each element separately in its own block of memory called a **"node"**.

✓ The list gets an overall structure by using **pointers** to connect all its nodes together.

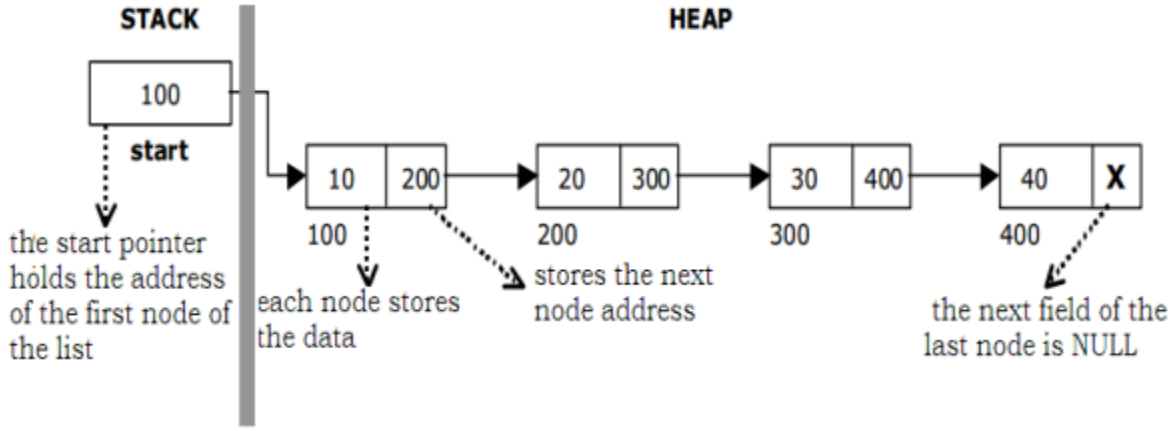
✓ Each node contains **two** fields - a **"data"** field to store element, and a **"next"** field which is a pointer used to connect to the next node.

✓ Each node is allocated in the **heap** using **malloc()** and it is explicitly de-allocated using **free().**

✓ The single linked list starts with a pointer to the **"start"** node.

✓ The single linked list is called as **linear list** or chain.

✓ The **traversing** of data can be in **one** direction only.



Single Linked List Representation

✓The beginning of the linked list is stored in a **"start"** pointer which points to the first node.

✓ The first node contains a pointer to the second node. The second node contains a pointer to the third node and so on.

✓ The last node in the list has its next field set to **NULL** to mark the end of the list.

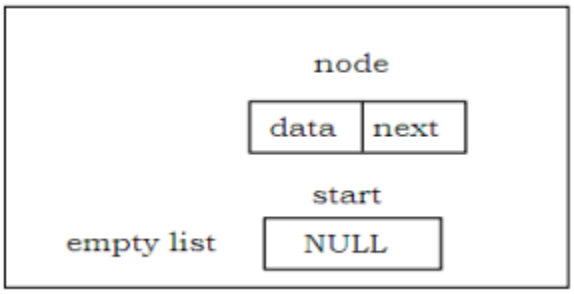
IMPLEMENTATION OF SINGLE LINKED LIST:

Before writing the code to build the list, we need to create a start node, used to create and access other nodes in the linked list.

✓ Creating a structure with one data item and a next pointer, which will be pointing to next node of the list. This is called as self-referential structure.

✓ Initialize the start pointer to be NULL.

```
struct slinklist
{
        int data;
        struct slinklist* next;
};
typedef struct slinklist node;
node *start = NULL;
```



BASIC OPERATION PERFORMED ON SINGLE LINKED LIST:

The different operations performed on the single linked list are listed as follows.

1. Creation 2. Insertion 3. Deletion 4. Traversing 5. Searching

Creating a node for Single Linked List:

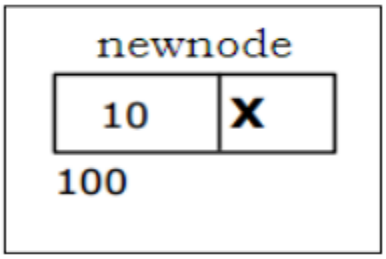
✓ Creating a singly linked list starts with creating a node.

✓ Sufficient memory has to be allocated for creating a node.

✓The information is stored in the memory, allocated by using the **malloc()** function.

✓ The function **getnode()**, is used for creating a node, after allocating memory for the node, the information for the node data part has to be read from the user and set next field to **NULL** and finally return the node.

```
node* getnode()
{
        node* newnode;
        newnode = malloc(node);
        printf("Enter data");
        scanf("%d", &newnode -> data;
        newnode -> next = NULL;
        return newnode;
 }
```



Creating a Singly Linked List with 'n' number of nodes:

The following steps are to be followed to create 'n' number of nodes.

1. Get the new node using getnode().

```
newnode = getnode();
```

2. If the list is empty, assign new node as start.

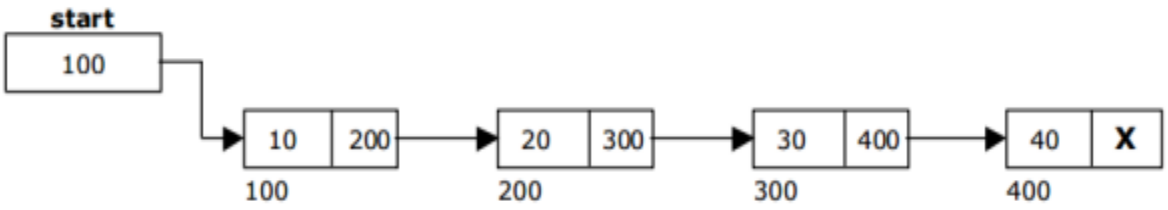
```
start = newnode;
```

3. If the list is not empty, follow the steps given below.

✓ The next field of the new node is made to point the first node (i.e.start node) in the list by assigning the address of the first node.

✓ The start pointer is made to point the new node by assigning the address of the new node.

4. Repeat the above steps 'n' times.



Single Linked List with 4 nodes

The function createlist(), is used to create 'n' number of nodes

```
void createlist(int n)
{
        int i;
        node *newnode;
        node *temp;
        for(i = 0; i < n ; i++)
        {
                newnode = getnode();
                if(start = = NULL)
                {
                        start = newnode;
                }
        else
        {
                temp = start;
                while(temp -> next != NULL)
                temp = temp -> next;
                temp -> next = newnode;
        }
} }
```

INSERTION OF A NODE:

✓ One of the most important operations that can be done in a singly linked list is the insertion of a node.

✓ Memory is to be allocated for the new node before reading the data.

✓ The new node will contain empty data field and empty next field.

✓ The data field of the new node is then stored with the information read from the user.

✓ The next field of the new node is assigned to NULL.

✓ The new node can then be inserted at three different places namely:

- ✓**Inserting a node at the beginning.**
- ✓ **Inserting a node at the end.**
- ✓ **Inserting a node at specified position.**

INSERTING A NODE AT THE BEGINNING:

The following steps are to be followed to insert a newnode at the beginning of the list:

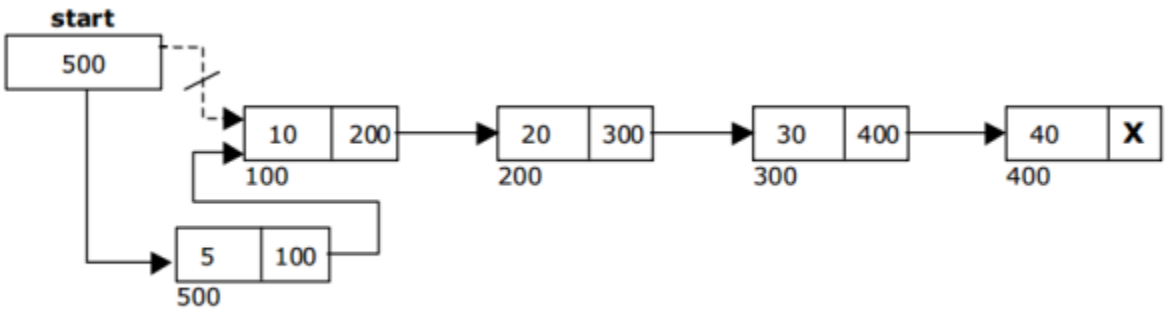
1. Get the newnode using getnode() then newnode = getnode();

2. If the list is empty then start = newnode.

3. If the list is not empty, follow the steps given below:

newnode -> next = start;

start = newnode;



Inserting a node at the begining of the list

The function insert_at_beg(), is used for inserting a node at the beginning.

```
void insert_at_beg()
{
        node *newnode;
        newnode = getnode();
        if(start == NULL)
        {
                start = newnode;
        }
        else
        {
                newnode -> next = start;
                start = newnode;
        }
}
```

INSERTING A NODE AT THE END:

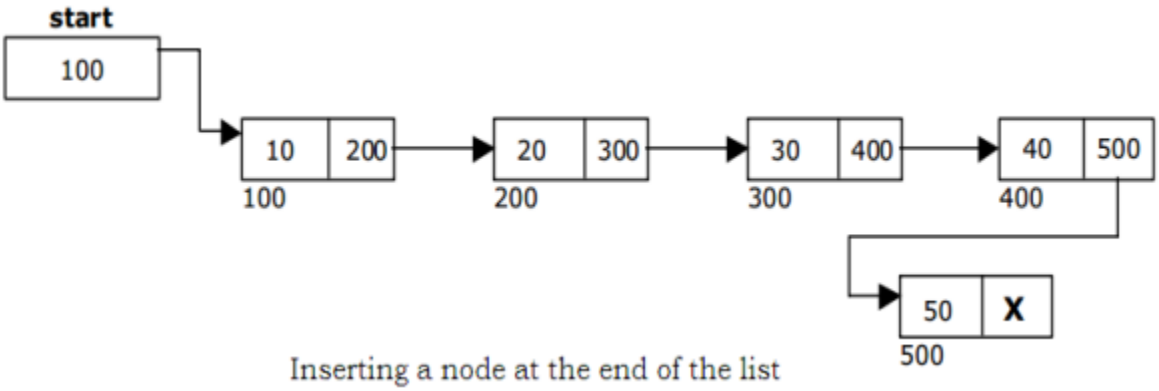
The following steps are followed to insert a new node at the end of the list:

1. Get the new node using getnode() then newnode = getnode();

2. If the list is empty then start = newnode.

3. If the list is not empty follow the steps given below:

```
temp = start;
while(temp -> next != NULL)
temp = temp -> next;
temp -> next = newnode;
```



Inserting a node at the end of the list

The function insert_at_end(), is used for inserting a node at the end.

```
void insert_at_end()
{
        node *newnode, *temp;
        newnode = getnode();
        if(start == NULL)
        {
                start = newnode;
        }
        else
        {
                temp = start;
                while(temp -> next != NULL)
                temp = temp -> next;
                temp -> next = newnode;
        }
}
```

INSERTING A NODE AT SPECIFIED POSITION:

The following steps are followed, to insert a new node in an intermediate position in the list:

1. Get the new node using getnode() then newnode = getnode();

2. Ensure that the specified position is in between first node and last node.

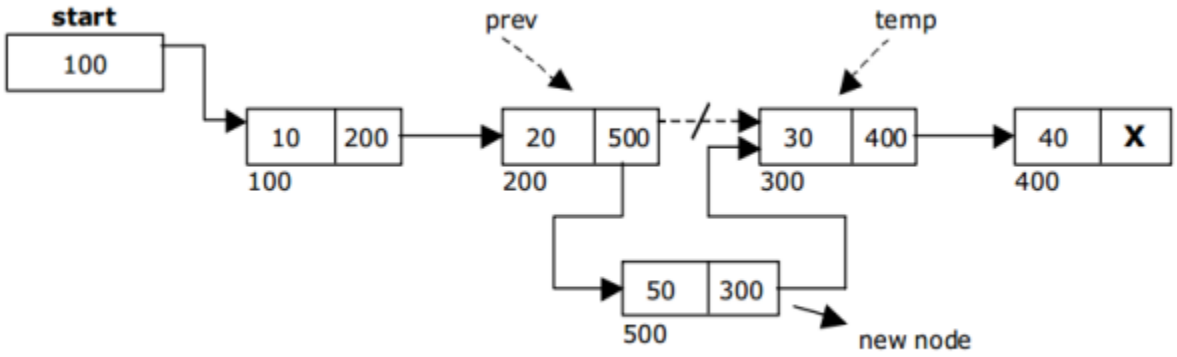
If not, specified position is invalid. This is done by countnode() function.

3. Store the starting address (which is in start pointer) in temp and prev pointers. Then traverse the temp pointer upto the specified position followed by prev pointer.

4. After reaching the specified position, follow the steps given below:

prev -> next = newnode;

newnode -> next = temp;



Inserting a node at specified position

The function insert_at_mid(), is used for inserting a node in the intermediate position.

```
void insert_at_mid()
{
        node *newnode, *temp, *prev;
        int pos, nodectr, ctr = 1;
        newnode = getnode();
        printf(" Enter the position");
        scanf("%d", &pos);
        nodectr = countnode(start);
        if(pos > 1 && pos < nodectr)
        {
                temp = prev = start;
                while(ctr < pos)
                {
                        prev = temp;
                        temp = temp -> next;
                        ctr++;
                }
                prev -> next = newnode;
                newnode -> next = temp;
        }
        else
        {
```

```
            printf("%d", pos);
        }
}
```

DELETION OF A NODE:

✓ Another operation that can be done in a singly linked list is the deletion of a node.

✓ Memory is to be released for the node to be deleted.

✓ It is done by using free() function.

✓ A node can be deleted from the list from three different places.

✓ Deleting a node at the beginning.

✓ Deleting a node at the end.

✓ Deleting a node at specified position

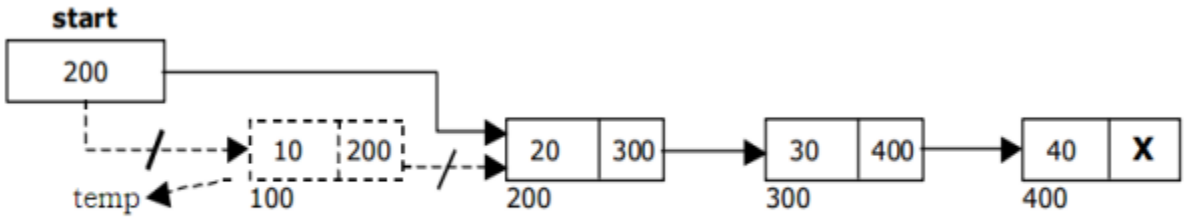
DELETING A NODE AT THE BEGINNING:

The following steps are followed, to delete a node at the beginning of the list:

1. If list is empty then display 'Empty List' message.

2. If the list is not empty, follow the steps given below:

```
temp = start;
start = temp -> next;
free(temp);
```



deleting a node at the begining

The function delete_at_beg(), is used for deleting the first node in the list.

```
void delete_at_beg()
{
        node *temp;
        if(start == NULL)
        {
                printf(" Empty List ");
                return ;
```

```
        }
        else
        {
                temp = start;
                start = temp -> next;
                free(temp);
                printf("Node deleted");
        }
}
```

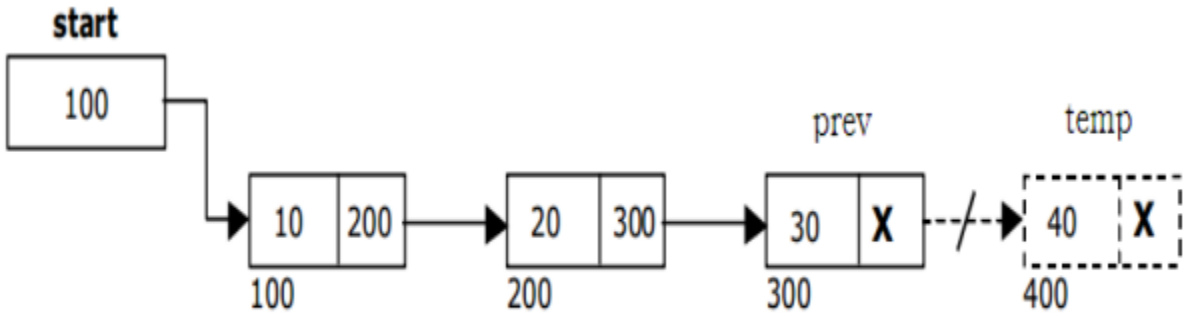
DELETING A NODE AT THE END:

The following steps are followed to delete a node at the end of the list:

1. If list is empty then display 'Empty List' message.

2. If the list is not empty, follow the steps given below:

```
        temp = prev = start;
        while(temp -> next != NULL)
        {
                prev = temp;
                temp = temp -> next;
        }
        prev -> next = NULL;
        free(temp);
```



deleting a node at the end

The function delete_at_last(), is used for deleting the last node in the list.

```
void delete_at_last()
{
        node *temp, *prev;
        if(start == NULL)
        {
                printf(" Empty List ");
                return ;
        }
        else
        {
```

```
                temp = start;
                prev = start;
                while(temp -> next != NULL)
                {
                        prev = temp;
                        temp = temp -> next;
                }
                prev -> next = NULL;
                free(temp);
                printf("Node deleted");
        }
}
```

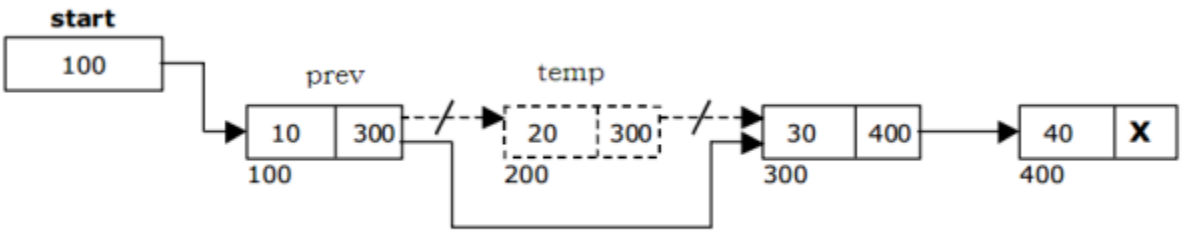
DELETING A NODE AT SPECIFIED POSITION:

The following steps are followed, to delete a node from the specified position in the list.

1. If list is empty then display 'Empty List' message

2. If the list is not empty, follow the steps given below.

```
        if(pos > 1 && pos < nodectr)
        {
                temp = prev = start;
                ctr = 1;
                while(ctr < pos)
                {
                        prev = temp;
                        temp = temp -> next;
                        ctr++;
                }
                prev -> next = temp -> next;
                free(temp);
                printf("Node deleted");
}
```



deleting a node at the specified position

The function delete_at_mid(), is used for deleting the specified position node in the list.

```
void delete_at_mid()
{
        int ctr = 1, pos, nodectr;
        node *temp, *prev;
        if(start == NULL)
```

```
        {
                printf(" Empty List ");
                return ;
        }
        else
        {
                printf(" Enter position of node to delete ");
                scanf("%d", &pos);
                nodectr = countnode(start);
                if(pos > nodectr)
                {
                        printf(" This node doesnot exist: ");
                }
        if(pos > 1 && pos < nodectr)
        {
                temp = prev = start;
                while(ctr < pos)
                {
                        prev = temp;
                        temp = temp -> next;
                        ctr ++;
                }
        prev -> next = temp -> next;
        free(temp);
        printf("Node deleted");
        }
        else
        printf("Invalid position");
        }
}
```

TRAVERSAL AND DISPLAYING A LIST (LEFT TO RIGHT):

To display the information, you have to traverse (move) a linked list, node by node from the first node, until the end of the list is reached.

Traversing a list involves the following steps.

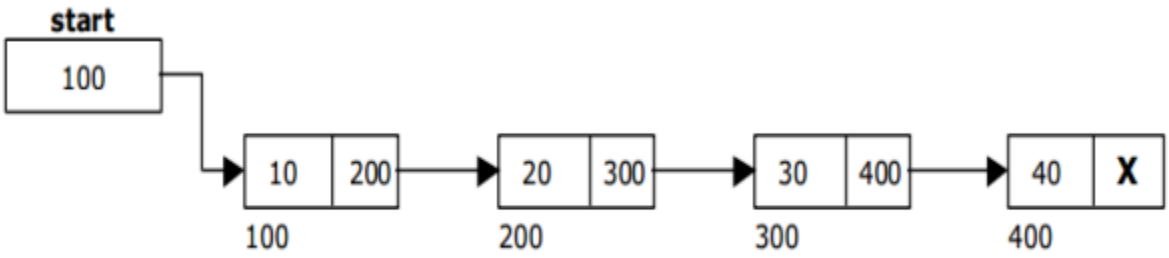
1. Assign the address of start pointer to a temp pointer.

2. Display the information from the data field of each node.

The function **traverse()** is used for traversing and displaying the information stored in the list from left to right.

```
void traverse()
{
        node *temp;
        temp = start;
        printf(" The contents of List (Left to Right) ");
        if(start == NULL )
```

```
        printf(" Empty List ");
        else
        {
        while (temp != NULL)
        {
                printf("%d", temp -> data);
                temp = temp -> next;
        }
}
}
```



Single Linked List with 4 nodes

SEARCHING A NODE IN A SINGLE LINKED LIST:

✓ Searching a single linked list means to find a particular element in the single linked list.

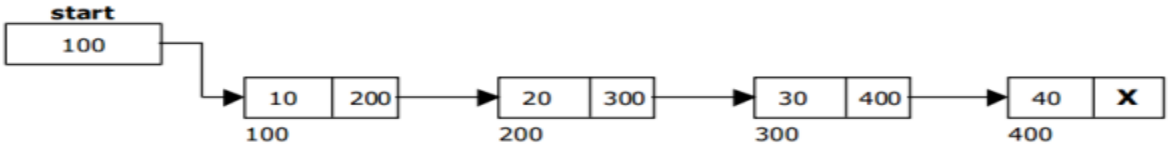
✓ A single linked list consists of nodes which are divided into two parts, the data part and the next part.

✓ So searching means finding whether a given value is present in the data part of the node or not.

✓ If it is present, then display element found otherwise element not found.

```
void search()
{
        node *temp;
        int value = 30;
        temp = start;
        if(start == NULL )
        printf(" Empty List ");
        else
        {
        while (temp != NULL)
        {
                if(value = temp->data)
                {
                        printf(" Element found ");
                        return;
                }
                temp = temp -> next;
        }
```

```
        printf(" Element not found ");
}   }
```

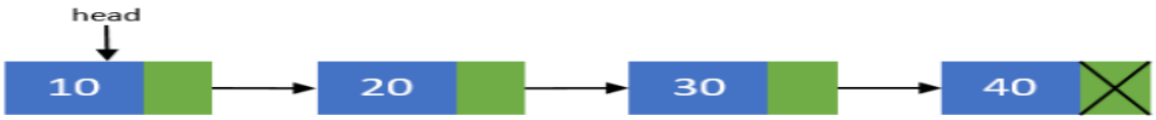


Single Linked List with 4 nodes

REVERSING SINGLE LINKED LIST:

✓ To reverse a single linked list containing nodes, first we have to create the single linked list with "n" nodes.

✓ For example the diagrammatic representation of single linked list with 4 nodes is as follows.



✓ After reversing the diagrammatic representation of single linked list with 4 nodes is as follows.

Algorithm to Reverse a Single Linked List:

```
if(head != NULL) then
        prevNode = head;
        head = head->next;
        curNode = head;
        prevNode->next = NULL;
while(head!=NULL)
{
        head = head->next;
        curNode->next = prevNode;
        prevNode = curNode;
        curNode = head;
}
head = prevNode;
```

APPLICATIONS ON SINGLE LINKED LIST:

POLYNOMIALS:

A polynomial is of the form

$$\sum_{i=0}^{n} c_i \, x^i$$

Where, ci is the coefficient of the ith term and n is the degree of the polynomial. Some examples are:

5x2 + 3x + 1

12x3 + 4

4x6 + 10x4 – 5x + 3

5x4 – 8x3 + 2x2 + 4x1 + 9

23x9 + 18x7 – 41x6 + 163x4 – 5x + 3

REPRESENTATION OF POLYNOMIALS:

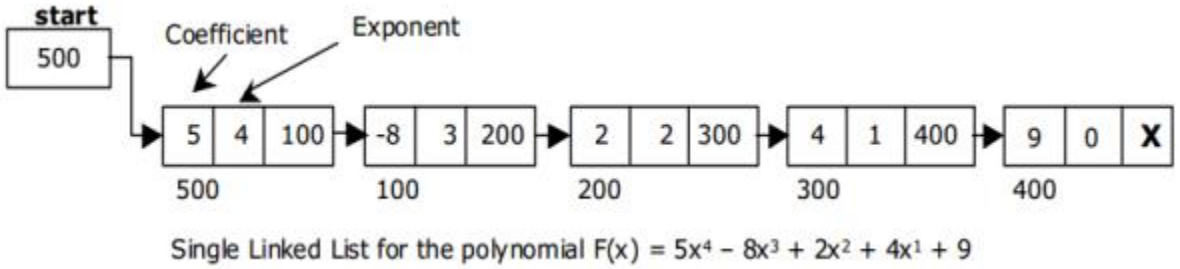
✓ It is not necessary to write terms of the polynomials in decreasing order of degree.

✓ In other words the two polynomials 1 + x and x + 1 are equivalent.

✓ The computer implementation requires implementing polynomials as a list of pairs of coefficient and exponent.

✓ Each of these pairs will constitute a structure, so a polynomial will be represented as a list of structures.

✓ A linked list structure that represents polynomials 5x4 – 8x3 + 2x2 + 4x1 + 9



Single Linked List for the polynomial $F(x) = 5x^4 - 8x^3 + 2x^2 + 4x^1 + 9$

Advantages :

✓ Save space ✓ Easy to maintain ✓ Do not need to allocate memory size initially

Disadvantages:

✓ It is difficult to back up to the start of the list

✓ It is not possible to jump to the beginning of the list from the end of the list

DOUBLY LINKED LISTS:

✓ A double linked list is a two-way list in which all nodes will have two links.

✓ This helps in accessing both successor node and predecessor node from the given node position.

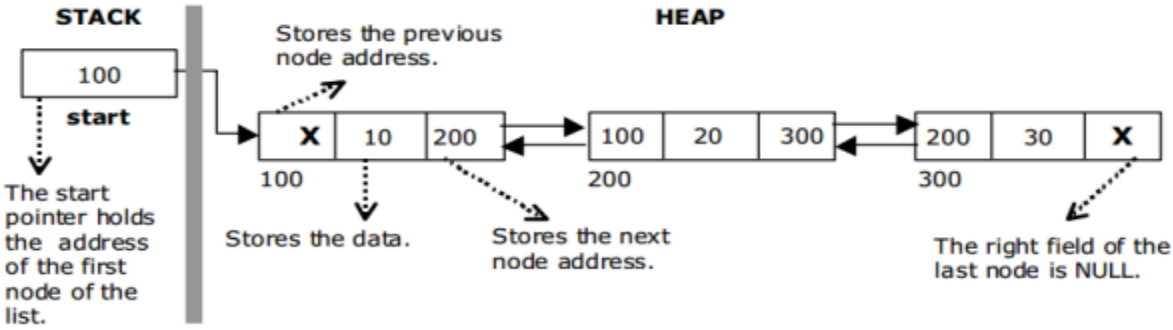
✓ It provides bi-directional traversing.

✓ Each node has three fields namely

- ✓ **Left link**
- ✓ **Data**
- ✓ **Right link**

✓ The left link points to the predecessor node and the right link points to the successor node. The data field stores the required data. The beginning of the double linked list is stored in a "start" pointer which points to the first node.

✓ The first node's left link and last node's right link is set to NULL.



Doubly Linked List Representation

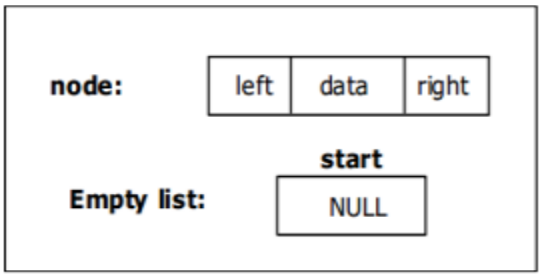
IMPLEMENTATION OF DOUBLY LINKED LIST:

Before writing the code to build the list, we need to create a start node, used to create and access other nodes in the linked list.

✓ Creating a structure with one data item and a right pointer, which will be pointing to next node of the list and left pointer pointing to the previous node. This is called as self-referential structure.

✓ Initialize the start pointer to be NULL

```
struct dlinklist
{
        struct dlinklist * left;
        int data;
        struct dlinklist * right;
};
```



typedef struct dlinklist node;
node *start = NULL;

BASIC OPERATION PERFORMED ON DOUBLY LINKED LIST:

The different operations performed on the doubly linked list are listed as follows.

1. Creation
2. Insertion
3. Deletion
4. Traversing
5. Display

Creating a node for Doubly Linked List:

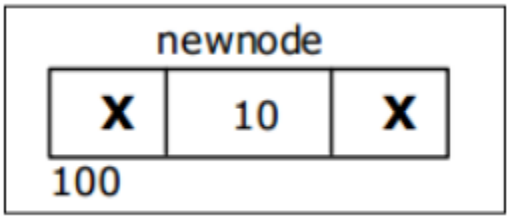
✓ Creating a double linked list starts with creating a node.

✓ Sufficient memory has to be allocated for creating a node.

✓ The information is stored in the memory, allocated by using the malloc() function.

✓ The function getnode(), is used for creating a node, after allocating memory for the node, the information for the node data part has to be read from the user and set left and right fields to NULL and finally return the node.

```
node* getnode()
{
        node* newnode;
        newnode = malloc(node);
        printf(" Enter data ");
        scanf("%d", &newnode -> data);
        newnode -> left = NULL;
        newnode -> right = NULL;
        return newnode;
}
```



Creating a Doubly Linked List with 'n' number of nodes:

The following steps are to be followed to create 'n' number of nodes.

1. Get the new node using getnode().

newnode = getnode();

2. If the list is empty, assign new node as start.

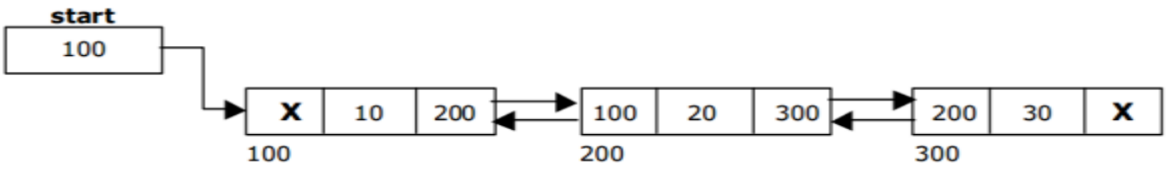
start = newnode;

3. If the list is not empty, follow the steps given below.

✓ The left field of the new node is made to point the previous node.

✓ The previous nodes right field must be assigned with address of the new node.

4. Repeat the above steps 'n' times.



Double Linked List with 3 nodes

The function createlist(), is used to create 'n' number of nodes

```
void createlist(int n)
{
        int i;
        node *newnode;
        node *temp;
        for(i = 0; i < n ; i++)
        {
                newnode = getnode();
                if(start = = NULL)
                {
                        start = newnode;
                }
                else
                {
                        temp = start;
                        while(temp -> right != NULL)
                        {
                                temp = temp -> right;
                        }
                temp -> right = newnode;
                newnode -> left = temp;
        }
        }
}
```

INSERTION OF A NODE:

✓ One of the most important operation that can be done in a doubly linked list is the insertion of a node.

✓ Memory is to be allocated for the newnode before reading the data.

✓ The newnode will contain empty data field and empty left and right fields.

✓ The data field of the newnode is then stored with the information read from the user.

✓ The left and right fields of the newnode are set to NULL.

✓ The newnode can then be inserted at three different places namely:

- ✓ **Inserting a node at the beginning.**
- ✓ **Inserting a node at the end.**
- ✓ **Inserting a node at specified position.**

INSERTING A NODE AT THE BEGINNING:

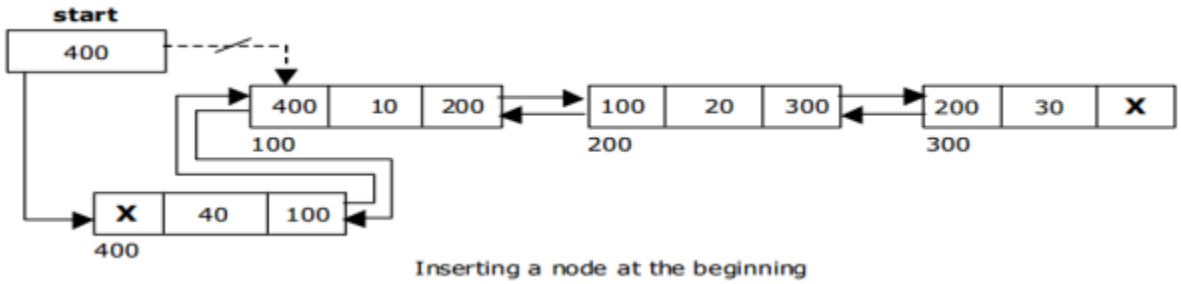
The following steps are to be followed to insert a newnode at the beginning of the list:

1. Get the newnode using getnode() then newnode = getnode();

2. If the list is empty then start = newnode.

3. If the list is not empty, follow the steps given below:

```
newnode -> right = start;
start -> left = newnode;
start = newnode;
```



Inserting a node at the beginning

INSERTING A NODE AT THE END:

The following steps are followed to insert a new node at the end of the list:

1. Get the new node using getnode() then newnode = getnode();

2. If the list is empty then start = newnode.

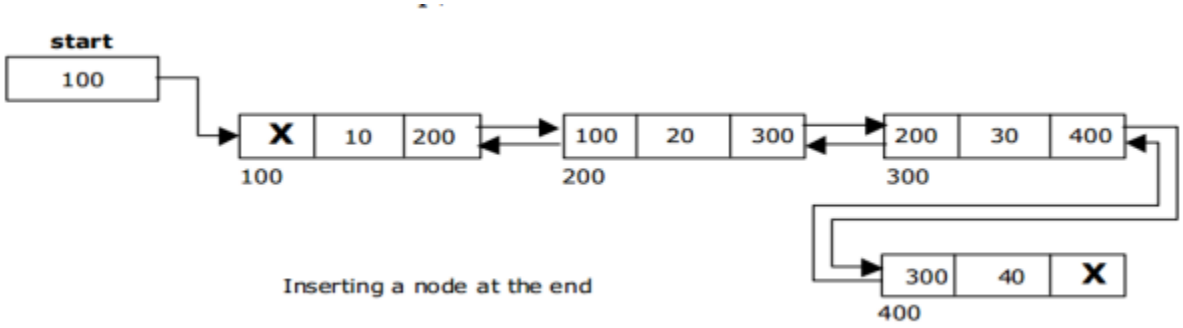
3. If the list is not empty follow the steps given below:

```
temp = start;
while(temp -> right != NULL)
```

temp = temp -> right;

temp -> right = newnode;

newnode -> left = temp;



Inserting a node at the end

INSERTING A NODE AT SPECIFIED POSITION:

The following steps are followed, to insert a new node in an intermediate position in the list:

1. Get the new node using getnode() then newnode = getnode();

2. Ensure that the specified position is in between first node and last node.

If not, specified position is invalid. This is done by countnode() function.

3. Store the starting address (which is in start pointer) in temp and prev pointers. Then traverse the temp pointer upto the specified position followed by prev pointer.

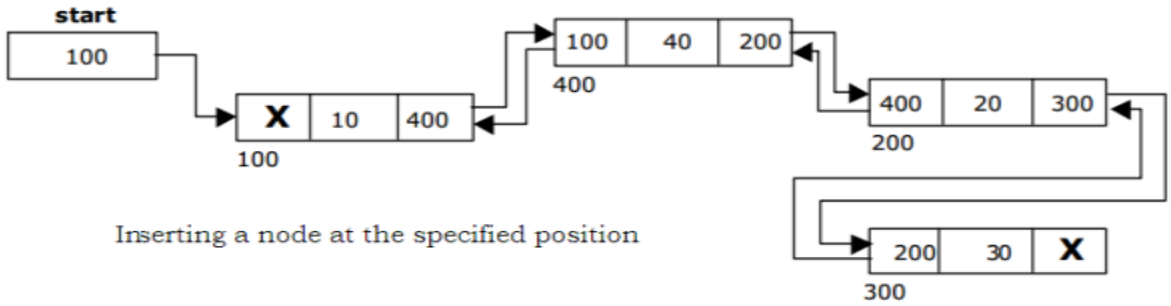
4. After reaching the specified position, follow the steps given below:

newnode -> left = temp;

newnode ->right = temp ->right;

temp -> right ->left = newnode;

temp -> right = newnode;



Inserting a node at the specified position

DELETION OF A NODE:

✓ Another operation that can be done in a doubly linked list is the deletion of a node.

✓ Memory is to be released for the node to be deleted.

✓ A node can be deleted from the list from three different places.

✓ Deleting a node at the beginning.

✓ Deleting a node at the end.

✓ Deleting a node at specified position.

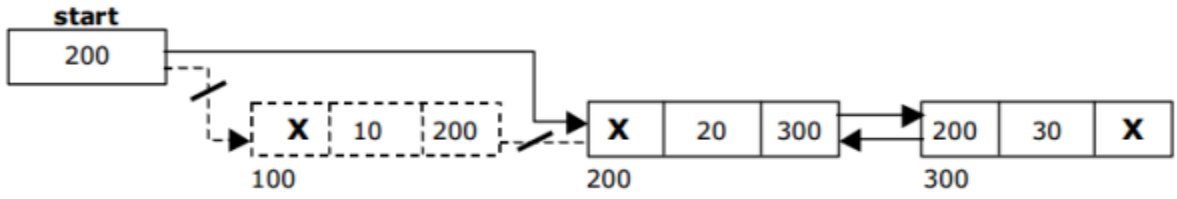
DELETING A NODE AT THE BEGINNING:

The following steps are followed, to delete a node at the beginning of the list:

1. If list is empty then display 'Empty List' message.

2. If the list is not empty, follow the steps given below:

```
temp = start;
start = temp -> right;
start -> left = NULL;
free(temp);
```



Deleting a node at beginning

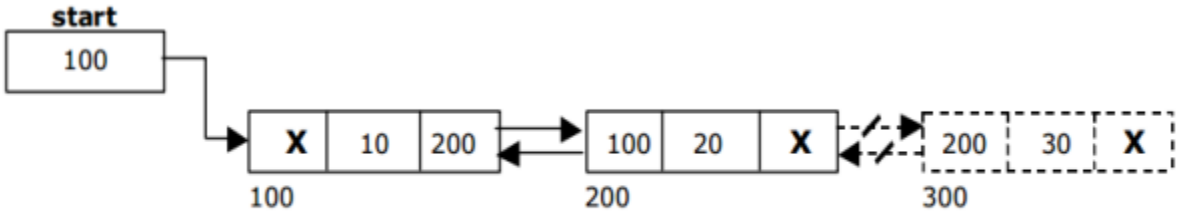
DELETING A NODE AT THE END:

The following steps are followed to delete a node at the end of the list:

1. If list is empty then display 'Empty List' message.

2. If the list is not empty, follow the steps given below:

```
temp = start;
while(temp -> right != NULL)
{
temp = temp ->right;
}
temp –> left -> right = NULL;
free(temp);
```



Deleting a node at the end

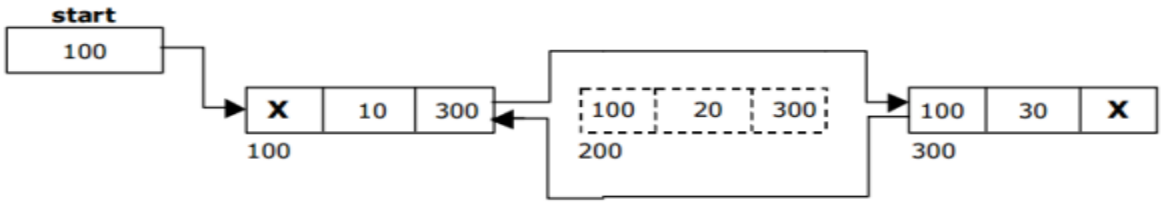
DELETING A NODE AT SPECIFIED POSITION:

The following steps are followed, to delete a node from the specified position in the list.

1. If list is empty then display 'Empty List' message

2. If the list is not empty, follow the steps given below.

```
if(pos > 1 && pos < nodectr)
{
        temp = start;
        ctr = 1;
        while(ctr < pos)
        {
                temp = temp -> right;
                ctr++;
        }
        temp -> right -> left = temp -> left;
        temp -> left -> right = temp -> right;
        free(temp);
}
```



Deleting a node at the specified position

TRAVERSAL AND DISPLAYING A LIST :

✓ To display the list, we have to traverse (move) the double linked list, node by node from the first node, until the end of the list is reached.

✓ To traverse double linked list from left to right we have the following steps:

1. If list is empty then display 'Empty List' message.

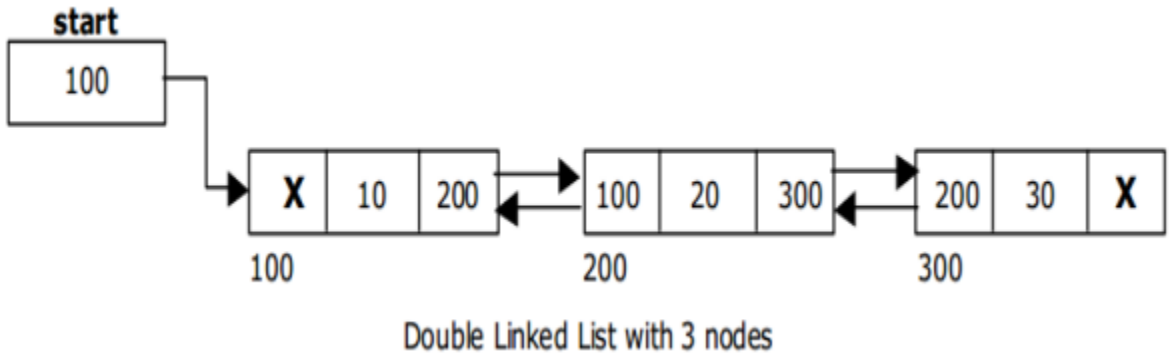
2. If the list is not empty, follow the steps given below:

```
temp = start;
```

```
while(temp != NULL)
{
printf("%d", temp -> data);
temp = temp -> right;
}
```

✓ To display the list, we have to traverse (move) the double linked list, node by node from the first node, until the end of the list is reached.

✓ The following steps are followed, to traverse a list from left to right:



Double Linked List with 3 nodes

COUNTING THE NUMBER OF NODES:

The following code will count the number of nodes exist in the list (using recursion).

```
int countnode(node *start)
{
if(start = = NULL)
return 0;
else
return(1 + countnode(start ->right ));
}
```

SEARCHING A NODE IN A DOUBLE LINKED LIST:

✓ Searching a double linked list means to find a particular element in the double linked list.

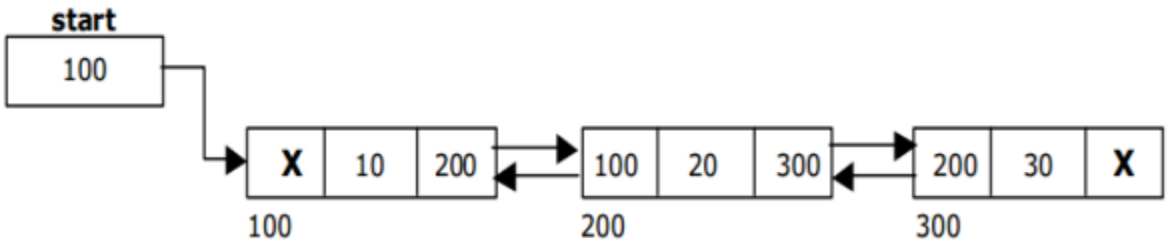
✓ A double linked list consists of nodes which are divided into two parts, the data part and the next part.

✓ So searching means finding whether a given value is present in the data part of the node or not.

✓ If it is present, then display element found otherwise element not found.

```
void search()
{
        node *temp;
        int value = 30;
        temp = start;
        if(start == NULL )
        printf(" Empty List ");
        else
        {
```

```
            while (temp->right != NULL)
            {
                  if(value = temp->data)
                  {
                        printf(" Element found ");
                        return;
                  }
                  temp = temp -> right;
            }
            printf(" Element not found ");
}}
```



Double Linked List with 3 nodes

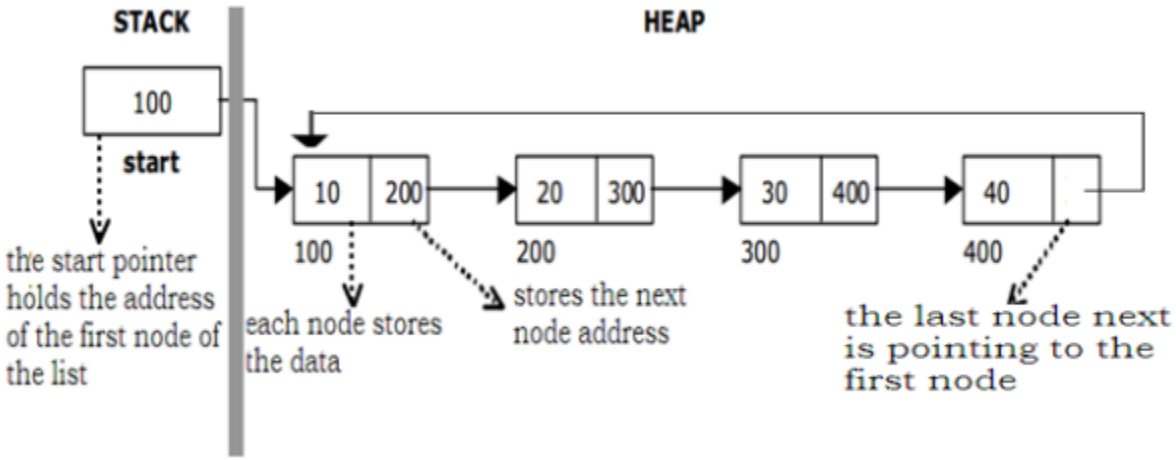
CIRCULAR LINKED LISTS:

✓ Circular linked list is a linked list which consists of collection of nodes each of which has two parts, namely the data part and the next part.

✓ The data part contains the value of the node and the next part has the address of the next node.

✓ The last node of list has the next pointer pointing to the first node thus making circular traversal possible in the list. A circular linked list has no beginning and no end.

✓ In circular linked list no null pointers are used, hence all pointers contain valid address



Circular Linked List Reresentation

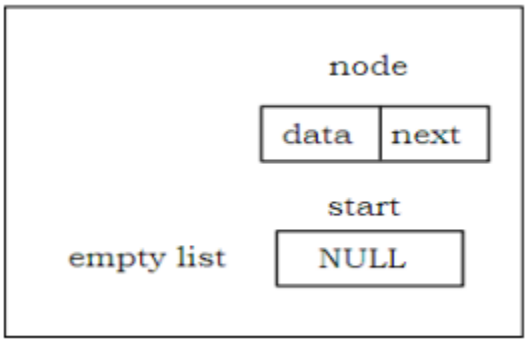
IMPLEMENTATION OF CIRCULAR LINKED LIST:

Before writing the code to build the list, we need to create a start node, used to create and access other nodes in the linked list.

✓ Creating a structure with one data item and a next pointer, which will be pointing to next node of the list. This is called as self-referential structure.

✓ Initialize the start pointer to be NULL.

```
struct clinklist
{
        int data;
        struct clinklist* next;
};
typedef struct clinklist node;
node *start = NULL;
```



BASIC OPERATION PERFORMED ON CIRCULAR LINKED LIST

The operations on the circular linked list are listed as follows.

1. Creation
2. Insertion
3. Deletion
4. Traversing
5. Display

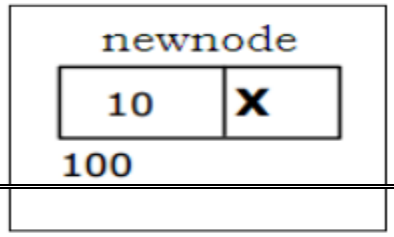
CREATING A NODE FOR CIRCULAR LINKED LIST:

✓ Creating a circular linked list starts with creating a node. Sufficient memory has to be allocated for creating a node.

✓ The information is stored in the memory, allocated by using the malloc() function.

✓ The function getnode(), is used for creating a node, after allocating memory for the node, the information for the node data part has to be read from the user and set next field to NULL and finally return the node.

```
node* getnode()
{
        node* newnode;
        newnode = malloc(node);
```



```
        printf(" Enter data ");
        scanf("%d", &newnode -> data);
        newnode -> next = NULL;
        return newnode;
}
```

Creating a Circular Linked List with 'n' number of nodes:

The following steps are to be followed to create 'n' number of nodes.

1. Get the new node using getnode().

```
        newnode = getnode();
```

2. If the list is empty, assign new node as start.

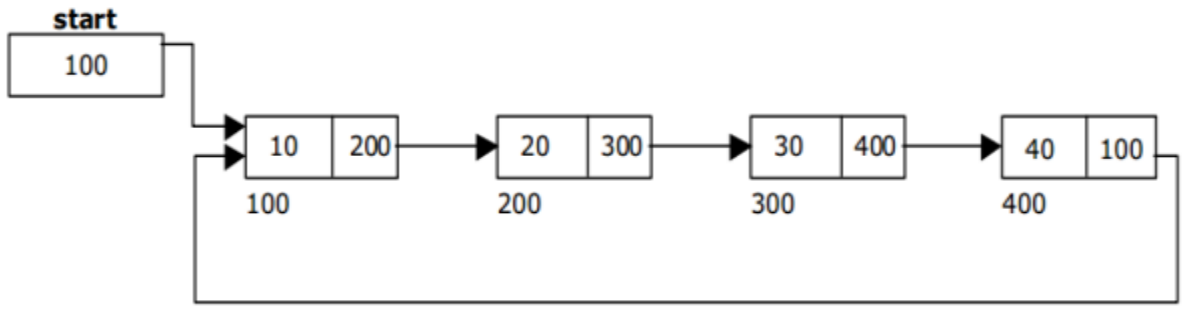
```
        start = newnode;
```

3. If the list is not empty, follow the steps given below.

```
        temp = start;
        while(temp -> next != Start)
        temp = temp -> next;
        temp -> next = newnode;
```

4. Repeat the above steps 'n' times.

5. newnode -> next = start;



Circular Linked List with 4 nodes

The function createlist(), is used to create 'n' number of nodes

```
void createlist(int n)
{
        int i;
        node *newnode;
        node *temp;
```

```
        for(i = 0; i < n ; i++)
        {
                newnode = getnode();
                if(start = = NULL)
                {
                        start = newnode;
                }
                else
                {
                        temp = start;
                        while(temp -> next != start)
                        temp = temp -> next;
                        temp -> next = newnode;
                }
                newnode -> next = start;
        }
}
```

INSERTING A NODE:

✓ One operation performed on circular linked list is the insertion of a node.

✓ Memory is to be allocated for the new node before reading the data.

✓ The new node will contain empty data field and empty next field. The data field of the new node is then stored with the information read from the user. The next field of the new node is assigned to NULL.

✓ The new node can then be inserted at three different positions:

✓ Inserting a node at the beginning.

✓ Inserting a node at the end.

INSERTING A NODE AT THE BEGINNING:

The following steps are to be followed to insert a new node at the beginning of the circular list:

1. Get the new node using getnode().

newnode = getnode();

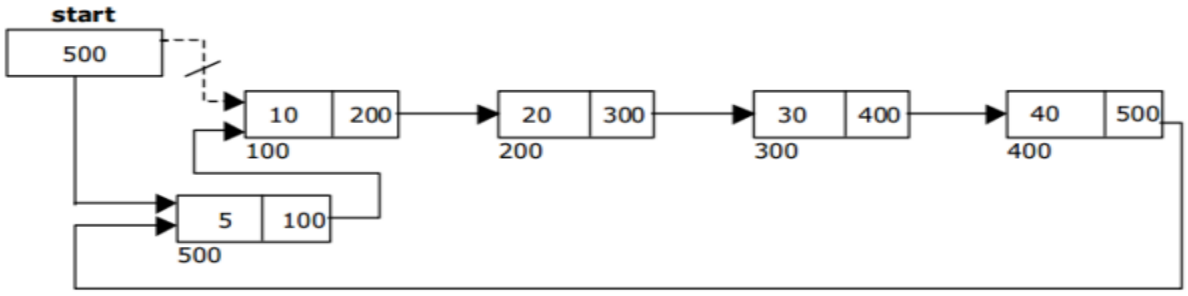
2. If the list is empty, assign new node as start.

start = newnode;

newnode -> next = start;

3. If the list is not empty, follow the steps given below:

```
last = start;
while(last -> next != start)
last = last -> next;
newnode -> next = start;
start = newnode;
last -> next = start;
```



Inserting a node at the beginning

INSERTING A NODE AT THE END:

The following steps are followed to insert a new node at the end of the list:

1. Get the new node using getnode().

newnode = getnode();

2. If the list is empty, assign new node as start.

start = newnode;

newnode -> next = start;

3. If the list is not empty follow the steps given below:

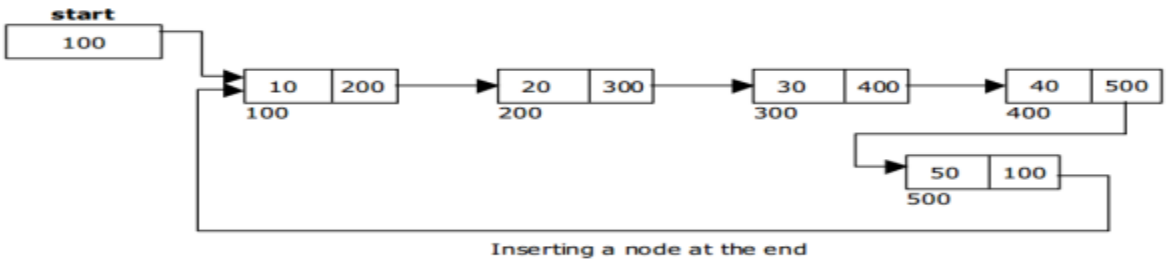
temp = start;

while(temp -> next != start)

temp = temp -> next;

temp -> next = newnode;

newnode -> next = start;



Inserting a node at the end

DELETING A NODE AT THE BEGINNING:

The following steps are followed, to delete a node at the beginning of the list:

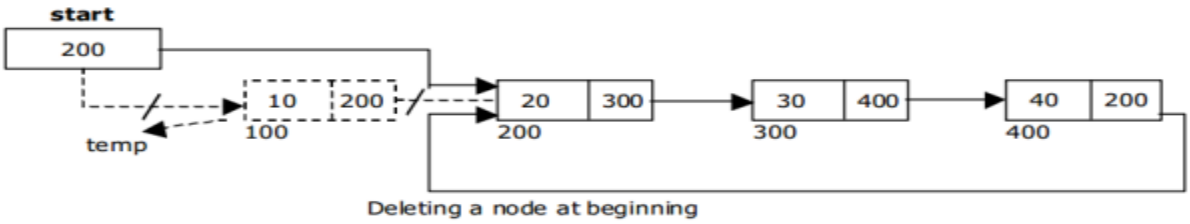
1. If the list is empty, display a message 'Empty List'.
2. If the list is not empty, follow the steps given below:

last = temp = start;

while(last -> next != start)

last = last -> next;
start = start -> next;
last -> next = start;

3. After deleting the node, if the list is empty then start = NULL.



Deleting a node at beginning

DELETING A NODE AT THE END:

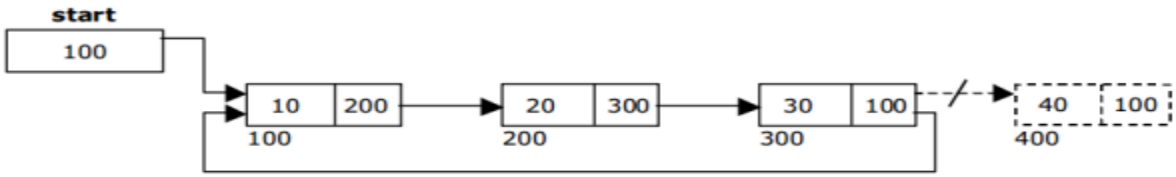
The following steps are followed to delete a node at the end of the list:

1. If the list is empty, display a message 'Empty List'.

2. If the list is not empty, follow the steps given below:

```
temp = start;
prev = start;
while(temp -> next != start)
{
        prev = temp;
        temp = temp -> next;
}
prev -> next = start;
```

4. After deleting the node, if the list is empty then start = NULL



Deleting a node at the end.

TRAVERSING A CIRCULAR LINKED LIST FROM LEFT TO RIGHT:

✓ To display the list, we have to traverse (move) the circular linked list, node by node from the first node, until the end of the list is reached.

✓ Traversing a list involves the following steps.

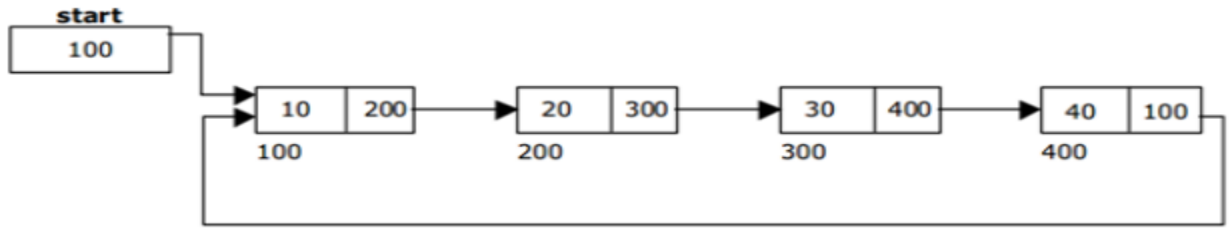
1. Assign the address of start pointer to a temp pointer.

2. Display the information from the data field of each node.

✓ The function traverse() is used for traversing and displaying the information stored in the list from left to right.

```
void traverse()
{
        node *temp;
```

```
        temp = start;
        printf(" The contents of List (Left to Right)");
        if(start == NULL )
        printf(" Empty List ");
        else
        {
        do
        {
                printf("%d", temp -> data);
                temp = temp -> next;
        } while (temp != start);
        }
}
```



Circular Linked List with 4 nodes

SEARCHING A NODE IN A CIRCULAR LINKED LIST:

✓ Searching a circular linked list means to find a particular element in the circular linked list.

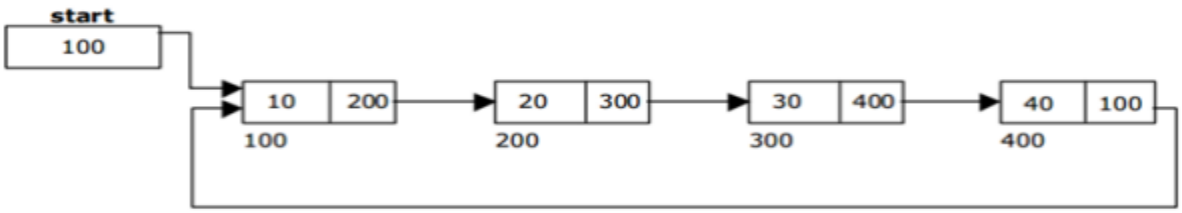
✓ A circular linked list consists of nodes which are divided into two parts, the data part and the next part.

✓ So searching means finding whether a given value is present in the data part of the node or not.

✓ If it is present, then display element found otherwise element not found.

```
void search()
{
        node *temp;
        int value = 30;
        temp = start;
        if(start == NULL )
        printf(" Empty List ");
        else
        {
                while (temp->next != start)
                {
                        if(value = temp->data)
                        {
                                printf(" Element found ");
                                return;
                        }
                        temp = temp -> next;
```

```
        }
        printf(" Element not found ");
} }
```



Circular Linked List with 4 nodes